

0.1 Soluciones a los problemas

Problema 1 Precondición:

$$\{x == 0 \wedge y == 0 \wedge n > 0\}$$

```

cobegin
  < x = x + 1 >   ||
  while (y < n) do
    begin
      < y = y + 1 > ;
    end
  coend

```

Poscondición:

$$\{x == 1 \wedge y == n\}$$

Demostración P_1 :

$$\{x == 0 \wedge y == 0 \wedge n > 0\} \quad \langle x = x + 1 \rangle \quad \{x == 1\}$$

Demostración P_2 :

- Invariante del bucle $I: 0 \leq y \leq n \wedge n > 0$: trivialmente, se cumple inicialmente.
- Aplicando la regla del WHILE, suponemos que se cumple $I \wedge y < n$ antes del cuerpo del WHILE y después se cumplirá: $y \leq n \wedge y \geq 0$
- En la terminación del WHILE, además se ha de cumplir: $\neg(y < n)$ y como también se cumple el invariante, entonces la poscondición de $P_2 \equiv y == n \wedge n > 0$
- La composición concurrente de P_1 y P_2 no presenta interferencia porque los procesos se refieren a las variables x e y por separado (variables *disjuntas*, no hay que demostrar teoremas de NI).
- Podemos concluir que la poscondición del programa es la conjunción de ambas poscondiciones, es decir: $x == 1 \wedge y == n$ c.q.d.

Problema 2 Variables globales:

```

int dentro = 0;           // procesos actualmente dentro
int esperando = 0;        // procesos esperando
sem_t mutex;
sem_t cola;               // suponemos sem foro con cola FIFO

```

Entrada:

```

void entrar() {
  sem_wait(&mutex);
  if (dentro < n) {
    dentro++;
    sem_post(&mutex);
  } else {
    esperando++;
    sem_post(&mutex);
    sem_wait(&cola);    // espera F I F O
  }
}

```

Salida:

```
void salir() {
    sem_wait(&mutex);
    dentro--;
    if (esperando > 0) {
        esperando--;
        dentro++;          // permite entrada directa
        sem_post(&cola);    // despierta solo a uno
    }
    sem_post(&mutex);
}
```

Problema 3:

```
Monitor CentroRecursos;
var
    dentro : integer;          // procesos dentro
    tipo_actual : integer;     // 0 = vacio, 1 2
    esperando : array[1..2] of integer;
    prioridad : array[1..2] of integer;
    cola : array[1..2] of condition;
procedure entrar(t : integer);
begin
    if (dentro = 0) or (tipo_actual = t) then
        begin
            dentro := dentro + 1;
            tipo_actual := t;
        end
    else
        begin
            esperando[t] := esperando[t] + 1;
            prioridad[t] := prioridad[t] + 1;
            cola[t].wait;
            esperando[t] := esperando[t] - 1;
            dentro := dentro + 1;
            tipo_actual := t;
        end;
    end;
end;
procedure salir(t : integer);
var other : integer;
begin
    dentro := dentro - 1;
    if dentro = 0 then
        begin
            other := 3 - t; // el otro tipo (1 2)
            if (esperando[t] > 0) and (prioridad[t] >= prioridad[other]) then
                cola[t].signal
            else if esperando[other] > 0 then
                cola[other].signal;
            end;
        end;
    end;
begin
    dentro := 0;
    tipo_actual := 0;
    esperando[1] := 0; esperando[2] := 0;
    prioridad[1] := 0; prioridad[2] := 0;
end;
```